

The Ultimate Blueprint: Building a No-Code AR SaaS Platform

A comprehensive engineering guide covering WebAR, Instant Apps, Unity Stripping, and Backend Microservices.

SECTION 1: Strategic Architecture Selection

As an AR engineering team, you must choose between two distinct distribution pathways for your clients:

Route A: The WebAR SaaS Model (e.g., MyWebAR)

- **How it works:** A QR code opens a standard website. The website requests camera access, downloads an asset, and renders AR directly in Safari/Chrome.
- **Pros:** Works on 100% of smartphones instantly. Easiest to develop. Easy to dynamically inject client assets.
- **Cons:** Browser performance is weaker than native. Users have to tap "Allow Camera" popups.

Route B: The Native "Instant App" Model (e.g., Flam)

- **How it works:** A QR code triggers the phone's OS to download a hyper-compressed (<50MB) "App Clip". It launches a buttery-smooth native camera experience from the lock screen.
 - **Pros:** Incredible native 60FPS tracking. Deeply impressive "magical" user experience. No browser UI.
 - **Cons:** Brutally hard to compress Unity below Apple's strict 50MB limit. Extremely complex DNS and Apple Developer Certificate management.
-

SECTION 2: Building The WebAR Backend Ecosystem

To automate your system, you effectively need three servers talking to each other.

2.1 The AWS S3 Storage & CORS Configuration

You *cannot* host client `.mp4` video files on your primary web server. You must use AWS S3 or Cloudflare R2. To allow a WebGL or WebAR canvas to read video pixels from an external CDN without throwing a **Tainted Canvas** security error, your S3 Bucket **must** have this exact CORS configuration:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "HEAD"],
    "AllowedOrigins": ["https://play.yourdomain.com",
"https://api.yourdomain.com"],
    "ExposeHeaders": ["Access-Control-Allow-Origin"]
  }
]
```

2.2 The PHP API Controller (Laravel Example)

Your PHP dashboard must act as the traffic controller. When a user uploads an image, PHP must save it to S3, call the Node.js compiler, and update MySQL.

```
<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\Http;
use App\Models\Campaign;

class ARCampaignController extends Controller
{
    public function store(Request $request)
    {
        // 1. Validate the user upload
        $request->validate([
            'image_marker' => 'required|image',
            'video_ad' => 'required|mimes:mp4,webm'
        ]);

        // 2. Upload raw files to AWS S3
        $imagePath = $request->file('image_marker')->store('raw_markers', 's3');
        $videoUrl = Storage::disk('s3')->url($request->file('video_ad')->store('videos', 's3'));

        // 3. Call Node.js Microservice to mathematically compile the .mind tracking file!
        $s3ImageUrl = Storage::disk('s3')->url($imagePath);
        $nodeResponse = Http::post('http://127.0.0.1:3000/compile', [
            'image_url' => $s3ImageUrl
        ]);

        $compiledTargetUrl = $nodeResponse->json()['target_url_s3'];

        // 4. Save the Campaign Data to MySQL
        $campaign = Campaign::create([
            'compiled_target_url' => $compiledTargetUrl,
            'video_url' => $videoUrl
        ]);

        return response()->json(['success' => true, 'qr_url' => 'https://play.yourdomain.com/?id=' . $campaign->id]);
    }
}
```

2.3 The Node.js Image Compiler Microservice

PHP cannot extract AI feature points from an image. You must run Node.js on port 3000 to handle the heavy mathematical tracking generation using [@mindar/image-compiler](#).

```
// server.js
const express = require('express');
const { Compiler } = require('@mindar/image-compiler');
const { loadImage } = require('canvas');
const fs = require('fs');
const AWS = require('aws-sdk');

const app = express();
app.use(express.json());
const s3 = new AWS.S3();

app.post('/compile', async (req, res) => {
  try {
    const imageUrl = req.body.image_url;

    // 1. Download image to memory
    const image = await loadImage(imageUrl);

    // 2. Compile AR tracking data
    const compiler = new Compiler();
    await compiler.compileImageTargets([image], (progress) => {
      console.log("Compiling: " + progress);
    });
    const exportedBuffer = await compiler.exportData();

    // 3. Upload compiled .mind file back to S3
    const upload = await s3.upload({
      Bucket: 'your-bucket-name',
      Key: `compiled_targets/target_${Date.now()}.mind`,
      Body: Buffer.from(exportedBuffer)
    }).promise();

    // 4. Return the new S3 URL back to PHP
    res.json({ target_url_s3: upload.Location });

  } catch (e) {
    res.status(500).send(e.toString());
  }
});

app.listen(3000, () => console.log('Node.js Compiler Microservice running on port 3000'));
```

SECTION 3: Executing Route B (The Unity "Flam" App Clip)

If you choose to ignore WebAR and want to build the "Flam" Native Instant App in Unity, here are the exact engineering steps to make your Unity App launch instantly from an iOS Camera QR Code.

3.1 Network DNS Setup (AASA)

To intercept a QR code and prevent Safari from opening, your web server ([yourdomain.com](#)) must host a file at <https://yourdomain.com/.well-known/apple-app-site-association>. **There is no .json extension on this file.**

```
{
  "appclips": {
    "apps": ["TEAMID.com.yourcompany.arapp.Clip"]
  }
}
```

3.2 Crushing the Unity Build Size (The 50MB Limit Rule)

Apple's App Clip limit is 50MB (uncompressed). A blank Unity ARKit project is usually 80MB. To force Unity into an App Clip:

1. **Disable Bitcode:** In iOS Player Settings, Bitcode inflates sizes massively. Turn it off.
2. **Managed Stripping Level:** Go to [Project Settings > Player > Optimization](#). Set [Managed Stripping Level](#) to **High**. This forces the Unity compiler to delete thousands of unused C# libraries.
3. **Delete Physics & UI:** If your AR App just plays videos, open Unity's [Package Manager](#) and completely uninstall the [Physics 2D](#), [Physics 3D](#), [UI Toolkit](#), and [Audio](#) packages.
4. **Texture Compression:** Change iOS Texture Override to [ASTC 8x8](#) format.
5. **IL2CPP Code Generation:** Set to [Faster \(Smaller\) Builds](#) instead of [Faster Runtime](#).
6. **No Embedded Tracks:** You cannot embed Videos inside Unity's [StreamingAssets](#). All videos must be dynamically streamed from [cdnVideoUrl](#) just like your [CDNARVideoController.cs](#) does.

3.3 Dynamic Target Retrieval in C#

Inside your tiny Unity App Clip, you still need to load what the client wants to see dynamically. Because an App Clip is triggered by a URL, Unity must read the URL that launched the app.

```
using UnityEngine;

public class AppClipManager : MonoBehaviour
{
    void Start()
    {
        // 1. Ask iOS what URL the user scanned via the QR code
        string scannedUrl = GetAppClipLaunchURL();

        // e.g. scannedUrl = "https://play.yourdomain.com/?id=892"
        string campaignId = ExtractIdFromUrl(scannedUrl);

        // 2. Query your PHP Server for Campaign 892

        StartCoroutine(FetchCampaignData("https://api.yourdomain.com/campaigns.php?id=" +
            campaignId));
    }
}
```

```
}

// You would use an iOS Native Plugin to fetch the NSUserActivity WebpageURL
private string GetAppClipLaunchURL() {
    // Pseudo-code implementation via iOS Unity Plugin bridging
    #if UNITY_IOS && !UNITY_EDITOR
        return IOSNativeMethods.GetLaunchURL();
    #else
        return "https://play.yourdomain.com/?id=test";
    #endif
}
}
```

SECTION 4: Final Development Playbook

If you want to start building this SaaS empire right this second, take these exact steps:

Phase 1: Validation (Week 1)

1. Do not use Unity yet. Copy the `index.html` A-Frame code from Version 1 of this document.
2. Test scanning a basic image using the MindAR WebAR engine on your mobile phone to prove that browser-based AR is viable for your clients.

Phase 2: Microservice API (Week 2)

1. Spin up a Virtual Private Server (VPS).
2. Write the small `server.js` Node script provided in Section 2.3.
3. Call it from Postman or your terminal with a dummy image to prove your backend can generate `.mind` tracker files on its own without you touching an editor.

Phase 3: The PHP Bridge (Week 3)

1. Rewrite your `upload.php` logic to hit the Node.js server.
2. Store the returned CDN URLs into your MySQL tables.

Phase 4: The Decision (Week 4) You now have a fully functional automated backend. You simply have to decide what your "Scanner Player" is:

- *Option A:* Use the A-Frame HTML file (Fastest, cheapest, easiest, universal).
- *Option B:* Spend 3 months creating an aggressively stripped, dynamically bridged Unity App Clip and fighting Apple/Google for App Clip approval (Highest risk, but beautiful "Flam" native quality).